Mapping between Gaussian and Uniform Distributions

Curtis Mellsop
Victoria University of Wellington

I. Introduction

Mapping between data distributions of differing types and dimensionality is a common challenge in machine learning. This report investigates three fully connected neural network architectures for transforming between Gaussian and uniform data distributions.

II. Theory

This section discusses the underlying principles used in the model creation.

A. Input data

Each model took in different types of data as inputs. The first model took two-dimensional zero-mean unit variance normal distributed (Gaussian) data:

$$Z \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{1}$$

The second model took two-dimensional uniform data, of edge length 1:

$$Y \sim \mathcal{U}([0,1] \times [0,1]) \tag{2}$$

The final model took one-dimensional Uniform data:

$$Y \sim \mathcal{U}(0,1) \tag{3}$$

B. Neural Network model descriptions

Three Neural Network models were created, trained, and tested:

a) Model 1: 2D Gaussian to 2D Uniform: The first model comprises four fully connected layers with sizes 2, 20, 20, and 2. It takes 2D Gaussian vectors as input and outputs the corresponding predicted 2D uniform vectors.

$$f : \mathbb{R}^2 \to \mathbb{R}^2, \quad Y = f(Z) \tag{4}$$

b) Model 2: 2D Uniform to 2D Gaussian: The second model comprises five fully connected layers with sizes 2, 50, 50, 50, and 2. It takes 2D uniform vectors as input and outputs the corresponding predicted 2D Gaussian vectors.

$$f : \mathbb{R}^2 \to \mathbb{R}^2, \quad Z = f(Y) \tag{5}$$

c) Model 3: 1D Uniform to 2D Gaussian: The third model comprises four fully connected layers with sizes 1, 50, 50 and 2. It takes 1D uniform scalars as input and outputs the corresponding predicted 2D Gaussian vectors.

$$f : \mathbb{R}^1 \to \mathbb{R}^2, \quad Z = f(Y) \tag{6}$$

All models used Leaky ReLU after each hidden layer; Model 2 additionally applied Sigmoid at the output to constrain values to [0, 1]. Models 1 and 3 were trained unsupervised with Maximum Mean Discrepancy loss, while Model 2 used supervised learning with mean squared error loss.

C. Loss Functions

a) Mean Squared Error: The mean squared error computes the average squared difference between the model's predicted and expected outputs.

$$L = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{y}_i - \hat{\mathbf{y}}_i\|^2 \tag{7}$$

This loss function is used in model two which has target values associated with input values (supervised).

b) Maximum Mean Discrepancy (MMD): The MMD function measures the loss by calculating the difference between the produced distribution and the target distribution.

$$\begin{aligned} \text{MMD}^2(X,Y) = &\frac{1}{m(m-1)} \sum_{i \neq j}^{m} k(x^{(i)}, x^{(j)}) \\ &+ \frac{1}{n(n-1)} \sum_{i \neq j}^{n} k(y^{(i)}, y^{(j)}) \\ &- \frac{2}{mn} \sum_{i=1}^{m} \sum_{j=1}^{n} k(x^{(i)}, y^{(j)}) \end{aligned} \tag{8}$$

MMD calculates pairwise similarities uses a kernel to calculate pairwise similarities between distributions. The kernel used in this case was the Gaussian Kernel:

$$k(\mathbf{x}, \mathbf{y}) = \exp\left(- \frac{\|\mathbf{x} - \mathbf{y}\|^2}{2\sigma^2} \right) \tag{9}$$

This loss function is used in models one and three where the output is a desired distribution.

D. Gradient Descent

Gradient descent iteratively updates model parameters to minimize the loss, using backpropagation to compute gradients. It is applied in all models, whether optimizing MSE for pointwise differences or MMD for distributional similarity.

E. Regularization

Regularization is a technique that reduces overfitting by adding a penalty to the loss, shrinking weights toward zero. Two main types exist:

a) Lasso Regularization (L1): L1 regularization uses the absolute values of the weights in the penalty term. This causes all weights to be pushed toward zero at a constant rate, regardless of magnitude. As a result, some weights become exactly zero, effectively removing the corresponding features from the model and performing implicit feature selection.

$$\Omega_{L1}(\mathbf{w}) = \lambda \sum_{i=1}^{n} |w_i| \tag{10}$$

b) Ridge Regularization (L2): L2 Regularization, on the other hand, uses squared values of the weights in the penalty term. This pushes weights towards zero at a smoother rate which is relative to their magnitude. Unlike L1 regularization, weights are never set to zero.

$$\Omega_{L2}(\mathbf{w}) = \lambda \sum_{i=1}^{n} w_i^2 \tag{11}$$

III. Experiments

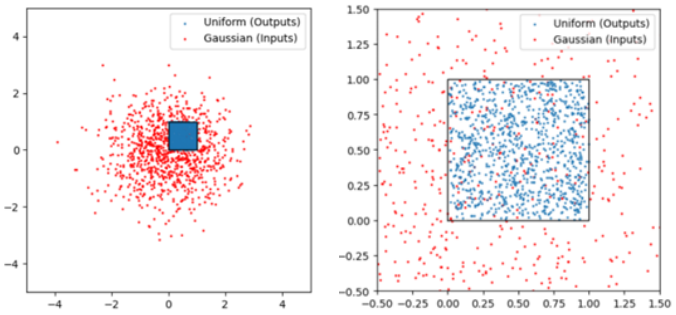
This section tests the performance of the three models based on their final loss, and the visualisation of their distributions. It also compares the effect of different regularization techniques on the weights of the first model

A. Setup

All models were trained for 5000 epochs. In each epoch, 256 input values were generated. For the second model, these inputs were produced by the first model, providing paired target values for supervised training. Whereas for the first and third model these inputs were generated randomly.

All models were trained using a learning rate of 0.001 with the Adam optimizer.

B. Performance of Model One



(a) Overall. (b) Zoomed.

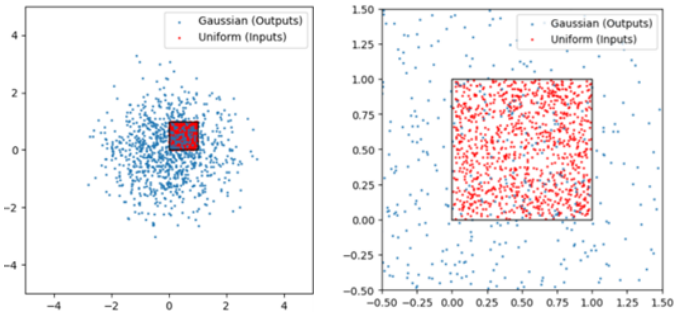
Fig. 1: Model one's inputs and outputs.

Figure 1 shows 1,000 randomly generated 2D Gaussian inputs (red) passed through the first model, along with the corresponding 2D uniform outputs (blue). The final MMD value of 0.0024 indicates that the predicted distribution closely matches the target uniform distribution.

C. Performance of Model Two

Figure 2 shows 1,000 randomly generated 2D uniform inputs (red) passed through the second model, along with the corresponding 2D Gaussian outputs (blue). The final MSE value of 0.0038 indicates that the model's predicted values very closely match the target values.

A more complex model, with additional layers and neurons, was used under supervised training as it produced the most accurate Gaussian outputs. Simpler models tended to generate outputs with a square-like pattern centered around the origin.



(a) Overall. (b) Zoomed.

Fig. 2: Model two's inputs and outputs

D. Effect of Regularization on weight values

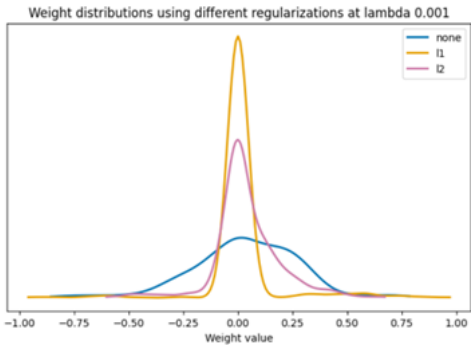


Fig. 3: Weight distributions using different regularization's.

Figure 3 shows the weight distributions of Model 1 under different regularization types ($\lambda = 0.001$). All distributions exhibit a roughly bell-shaped form. L2 regularization produces the highest peak (kurtosis), followed by L1, and then the unregularized model. These regularization techniques shrink weights toward zero, with L1 applying a constant linear penalty regardless of weight magnitude, resulting in it having the higher peak.

E. Performance of Model Three

The third model, intended to map 1D uniform data to 2D Gaussian data, instead produced outputs concentrated along a nonlinear curve through the origin, exhibiting a 2D uniform rather than 2D Gaussian distribution. This likely reflects the model's inability to introduce stochastic variation, which is essential for generating Gaussian-distributed outputs.

IV. Conclusion

The purpose of this work was to create and train three models capable of converting between different distributions - Gaussian and Uniform - and across different dimensions. Models one and two performed as intended; however, model three was unsuccessful due to its inability to incorporate randomness into the outputs.

STATEMENT

This report and the code provided were entirely my own work.

Tools used: Overleaf, Google Colab, PyTorch, NumPy, Matplotlib.

Source code is available from the following repository: https://github.com/CRM48/victoria_projects/tree/main/neural_networks/distribution_mapping.

It contains a ReadMe file with information on how the code works and how to use it. The ReadMe file also further discusses the failings of the third model.

It also contains the ipynb file which can be opened in Google Colab by clicking the link on the file.

The Neural Network model designs and training processes were inspired by my first projects model which was inspired by Rob Mulla [1].

REFERENCES

[1] R. Mulla, "Build Your First Pytorch Model In Minutes! [Tutorial + Code]," YouTube, Oct. 6, 2023. [Online]. Available: https://www.youtube.com/watch?v=tHL5STNJKag